

# Reviewer Pack — Noncommutative $L^p$ Geometry of Boolean Functions and Communi...

Entry 840377f38f9c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 14:34:23 UTC

## Noncommutative $L^p$ Geometry of Boolean Functions and Communication Complexity

Entry ID: 840377f38f9c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 14:34:23 UTC

### 1. Conjecture

**Field A** (mathematical branch): Noncommutative  $L^p$  Geometry **Field B** (complexity object): Communication Complexity

#### Statement:

[“For a given  $n$ -ary boolean function  $f$ , define the  $L^p$  norm of its noncommutative Fourier transform as  $\tau_p(f)$ . Then, for all  $n$  and all input pairs  $(x, y)$ ,  $\tau_p(f) \geq \Omega(n^{(1-p/k)})$  where  $k$  is the number of inputs in each party’s communication.”, ‘For all instances with  $|X|=|Y|=n \leq 40$ , if  $f$  is a boolean function that requires at least  $\Omega(n^{(1-p/k)})$  bits of randomized communication complexity to compute its value on an input pair  $(x, y)$ , then  $\tau_p(f) = \Theta(n^{(1-p/k)})$ .’]

#### Rationale (proposer’s reasoning):

[‘Noncommutative  $L^p$  geometry provides a framework for studying the spectral properties of boolean functions. By relating the  $L^p$  norm of the noncommutative Fourier transform to communication complexity, we may uncover new insights into the fundamental limits of computation.’, ‘This conjecture bridges the gap between two fields that have not been previously connected in the context of communication complexity, potentially leading to novel algorithms or hardness results.’]

**Taxonomy category:** COMM\_DISJ (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** d38ed4c56d8a9103

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if, for all  $n \leq 40$  and at least 100 random boolean functions  $f$ , the  $L^p$  norm  $\tau_p(f)$  correlates with communication complexity such that the correlation coefficient exceeds 0.9 and  $\tau_p(f) / n^{(1-p/k)} \geq 0.8$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - intitle:Noncommutative  $L^p$  Geometry AND boolean functions intitle:Communication Complexity - title: $L^p$  norm AND noncommutative Fourier transform AND communication complexity - randomized communication complexity AND boolean function AND  $\tau_p(f) = \theta(n^{(1-p/k)})$

**Top relevant hits considered:** - [http://arxiv.org/abs/1204.3878v1] Analytic constructions of p-adic L-functions and Eisenstein series - [http://arxiv.org/abs/1901.01957v6] ( $p$ -adic)  $L$ -functions and ( $p$ -adic) (multiple) zeta values - [http://arxiv.org/abs/2111.12700v2] Universality in long-distance geometry and quantum complexity - [http://arxiv.org/abs/2501.06483v2] Study of light-meson resonances decaying to  $K_S^0 K \pi$  in the  $B \rightarrow (K_S^0 K \pi) K$  channels - [http://arxiv.org/abs/1608.07537v3] Updated baseline for a staged Compact Linear Collider - [http://arxiv.org/abs/2505.04688v2] Euclid preparation. The impact of redshift interlopers on the two-point correlation function analysis

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90s$  wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def noncommutative_fourier_transform(f):
    n = len(f)
    transform = [0] * (1 << n)
    for j in range(1 << n):
        term = 0
        for i in range(n):
            if j & (1 << i):
                term += f[i]
            else:
                term -= f[i]
        transform[j] = Fraction(term, 2 ** n)
    return transform

def communication_complexity(f):
    n = len(f)
    max_cc = 0
    for x in range(1 << n):
```

```

    for y in range(1 << n):
        cc = 0
        for i in range(n):
            if (x & (1 << i)) != (y & (1 << i)):
                cc += 1
        max_cc = max(max_cc, cc)
    return max_cc

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = [random.choice([0, 1]) for _ in range(n)]

    transform = noncommutative_fourier_transform(f)
    cc = communication_complexity(f)

    tau_p = sum(abs(x) ** p for x in transform) ** (1 / p)
    expected_cc = n ** (1 - p / k)

    correlation_coefficient = (tau_p - expected_cc) / (n ** (1 - p / k))

    conjecture_holds = abs(correlation_coefficient) > 0.9 and tau_p / n ** (1 - p / k) >= 0.8
    counterexample = "" if conjecture_holds else f"CC({f})"

    return {
        "metric_name": "communication_complexity",
        "metric_value": correlation_coefficient,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean = sum(result["metric_value"] for result in results) / len(results)
    std = (sum((result["metric_value"] - mean) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a39a167b.py", line 73, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a39a167b.py", line 48, in run_trial
    transform = noncommutative_fourier_transform(f)
                ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a39a167b.py", line 20, in noncommutative_fourier_transform
    transform = [0] * (1 << n)
                ~~~~~^~~~~~
MemoryError
```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test with increased memory allocation or optimize the code to avoid crashing and verify if the conjecture's support conditions are met.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12008        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5866         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4850         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5929         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10743        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8650         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11240        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8903         |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8719         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 76909 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/840377f38f9c.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/840377f38f9c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/840377f38f9c.tar.gz` (if generated)